

Gaussian Elimination (and Numerics)

**Geometric Algorithms
Lecture 3**

Practice Problem

$$x + hy = 3$$

$$2x - 5y = k$$

For what values of h and k is the above system inconsistent?

Solution

$$\begin{aligned}x + hy &= 3 \\ 2x - 5y &= k\end{aligned}$$

Outline

- » Look at pseudocode of Gaussian Elimination
- » Discuss number representations, and look at the consequences of floating point representations
- » *If there's time:* Analyze the running time of Gaussian Elimination

Keywords

forward elimination

back substitution

floating point numbers

IEEE-754

relative error

`numpy.isclose`

ill-conditioned problems

Recap

Recap: Echelon Form

$$\begin{bmatrix} 0 & \blacksquare & * & * & * & * & * & * & * & * \\ 0 & 0 & 0 & \blacksquare & * & * & * & * & * & * \\ 0 & 0 & 0 & 0 & \blacksquare & * & * & * & * & * \\ 0 & 0 & 0 & 0 & 0 & \blacksquare & * & * & * & * \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \blacksquare & * \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$\blacksquare$ = nonzero, $*$ = anything

Recap: Echelon Form

next leading entry
to the right

$$\begin{bmatrix} 0 & \blacksquare & * & * & * & * & * & * & * & * \\ 0 & 0 & 0 & \blacksquare & * & * & * & * & * & * \\ 0 & 0 & 0 & 0 & \blacksquare & * & * & * & * & * \\ 0 & 0 & 0 & 0 & 0 & \blacksquare & * & * & * & * \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \blacksquare & * \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

all-zero rows at
the bottom

$\blacksquare$ = nonzero, $*$ = anything

Recap: Reduced Echelon Form

leading entries are 1

$$\begin{bmatrix} 0 & 1 & * & 0 & 0 & 0 & * & * & 0 & * \\ 0 & 0 & 0 & 1 & 0 & 0 & * & * & 0 & * \\ 0 & 0 & 0 & 0 & 1 & 0 & * & * & 0 & * \\ 0 & 0 & 0 & 0 & 0 & 1 & * & * & 0 & * \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & * \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

other column entries are 0

The diagram illustrates a 6x10 matrix in reduced echelon form. The matrix is enclosed in large square brackets. The entries are as follows: Row 1: [0, 1, *, 0, 0, 0, *, *, 0, *]; Row 2: [0, 0, 0, 1, 0, 0, *, *, 0, *]; Row 3: [0, 0, 0, 0, 1, 0, *, *, 0, *]; Row 4: [0, 0, 0, 0, 0, 1, *, *, 0, *]; Row 5: [0, 0, 0, 0, 0, 0, 0, 0, 1, *]; Row 6: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]. The leading entries (1s) are highlighted with light blue squares. Two blue arrows point from the text 'leading entries are 1' to the 1s in the second and fourth columns. Another blue arrow points from the text 'other column entries are 0' to the 0 in the eighth column of the fifth row.

Recap: The Fundamental Theorem

Recap: The Fundamental Theorem

Thm. *Every* matrix is row equivalent to a *unique* matrix in reduced echelon form

Recap: The Fundamental Theorem

Thm. *Every* matrix is row equivalent to a *unique* matrix in reduced echelon form

The Point. We can "read off" the solutions of a system of linear equations from its RREF

Recap: General Form Solution

Recap: General Form Solution

$$\begin{bmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

$$x_1 = 2 - x_3$$

$$x_2 = 1$$

x_3 is free

Recap: General Form Solution

$$\begin{bmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

$$x_1 = 2 - x_3$$

$$x_2 = 1$$

x_3 is free

1. For each pivot position (i,j) , isolate x_j in the equation in row i

Recap: General Form Solution

$$\begin{bmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

$$x_1 = 2 - x_3$$

$$x_2 = 1$$

x_3 is free

1. For each pivot position (i,j) , isolate x_j in the equation in row i

2. If x_i is not in a pivot column then write

x_i is free

Gaussian Elimination

A Word of Warning

A Word of Warning

The details of Gaussian elimination are tricky

A Word of Warning

The details of Gaussian elimination are tricky

The goal is not to understand it entirely

A Word of Warning

The details of Gaussian elimination are tricky

The goal is not to understand it entirely

***You should understand it well enough to use it
when solving by hand***

demo

Gaussian Elimination (Specification)

FUNCTION GE(A):

INPUT: $m \times n$ matrix A

OUTPUT: row equivalent $m \times n$ RREF matrix

...

Gaussian Elimination (High Level)

FUNCTION fwd_elim(A):

INPUT: $m \times n$ matrix A

OUTPUT: row equivalent $m \times n$ echelon form matrix

...

FUNCTION back_sub(A):

INPUT: $m \times n$ echelon form matrix A

OUTPUT: row equivalent $m \times n$ RREF matrix

...

FUNCTION GE(A):

RETURN back_sub(fwd_elim(A))

Elimination Stage

Elimination Stage (High Level)

Elimination Stage (High Level)

Input: matrix A of size $m \times n$

Output: echelon form of A

Elimination Stage (High Level)

Input: matrix A of size $m \times n$

Output: echelon form of A

Basic idea: starting top left and moving down,
find a leading entry and eliminate it from
latter equations

Example

Edge cases

Edge cases

What if the first equation doesn't have the variable x_1 ?

Edge cases

What if the first equation doesn't have the variable x_1 ?

Swap rows with an equation that does

Edge cases

What if the first equation doesn't have the variable x_1 ?

Swap rows with an equation that does

What if *none* of the equations have the variable x_1 ?

Edge cases

What if the first equation doesn't have the variable x_1 ?

Swap rows with an equation that does

What if *none* of the equations have the variable x_1 ?

Find the *leftmost* variable which appears in *any* of the remaining equations

Elimination Stage (Example)

$$\begin{bmatrix} 0 & 3 & -6 & 6 & 4 & -5 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 3 & -9 & 12 & -9 & 6 & 15 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 0 & 3 & -6 & 6 & 4 & -5 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 3 & -9 & 12 & -9 & 6 & 15 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 0 & 3 & -6 & 6 & 4 & -5 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 3 & -9 & 12 & -9 & 6 & 15 \end{bmatrix}$$

Swap R_1 and R_3

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

next entry
to zero

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

next entry
to zero

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

$$R_2 \leftarrow R_2 - R_1$$

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

swap R_2 with R_2

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

next entry
to zero

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

next entry
to zero

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

$$R_3 \leftarrow R_3 - \frac{3R_2}{2}$$

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

swap R_3 with R_3

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

done with elimination stage
going to back substitution stage

Elimination Stage (Pseudocode)

Elimination Stage (Pseudocode)

```
FUNCTION fwd_elim(A):
```


Elimination Stage (Pseudocode)

```
FUNCTION fwd_elim(A):
```

```
    FOR [i from 1 to m]: # for each row from top to bottom
```

Elimination Stage (Pseudocode)

```
FUNCTION fwd_elim(A):
```

```
    FOR [i from 1 to m]: # for each row from top to bottom
```

```
        IF [rows i...m are all-zeros]: # if remaining rows are zero
```


Elimination Stage (Pseudocode)

```
FUNCTION fwd_elim(A):
```

```
    FOR [i from 1 to m]: # for each row from top to bottom
```

```
        IF [rows i...m are all-zeros]: # if remaining rows are zero
```

```
            RETURN A
```

Elimination Stage (Pseudocode)

```
FUNCTION fwd_elim(A):
```

```
  FOR [i from 1 to m]: # for each row from top to bottom
```

```
    IF [rows i...m are all-zeros]: # if remaining rows are zero
```

```
      RETURN A
```

```
    ELSE:
```

Elimination Stage (Pseudocode)

FUNCTION fwd_elim(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [rows i...m are all-zeros]: # if remaining rows are zero

RETURN A

ELSE:

 (j, k) ← [position of leftmost entry in the rows i...m]

Elimination Stage (Pseudocode)

FUNCTION fwd_elim(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [rows i...m are all-zeros]: # if remaining rows are zero

RETURN A

ELSE:

 (j, k) ← [position of leftmost entry in the rows i...m]

 [swap row i and row j]

Elimination Stage (Pseudocode)

FUNCTION fwd_elim(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [rows i...m are all-zeros]: # if remaining rows are zero

RETURN A

ELSE:

 (j, k) ← [position of leftmost entry in the rows i...m]

 [swap row i and row j]

FOR [l from i + 1 to m]: # for all remaining rows

Elimination Stage (Pseudocode)

FUNCTION fwd_elim(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [rows i...m are all-zeros]: # if remaining rows are zero

RETURN A

ELSE:

 (j, k) ← [position of leftmost entry in the rows i...m]

 [swap row i and row j]

FOR [l from i + 1 to m]: # for all remaining rows

 [zero out A[l, k] using a replacement operation]

Elimination Stage (Pseudocode)

FUNCTION fwd_elim(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [rows i...m are all-zeros]: # if remaining rows are zero

RETURN A

ELSE:

 (j, k) ← [position of leftmost entry in the rows i...m]

 [swap row i and row j]

FOR [l from i + 1 to m]: # for all remaining rows

 [zero out A[l, k] using a replacement operation]

RETURN A

Elimination Stage (Example)

$$\begin{bmatrix} 0 & 3 & -6 & 6 & 4 & -5 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 3 & -9 & 12 & -9 & 6 & 15 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 0 & 3 & -6 & 6 & 4 & -5 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 3 & -9 & 12 & -9 & 6 & 15 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 0 & 3 & -6 & 6 & 4 & -5 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 3 & -9 & 12 & -9 & 6 & 15 \end{bmatrix}$$

Swap R_1 and R_3

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

next entry
to zero

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

next entry
to zero

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 3 & -7 & 8 & -5 & 8 & 9 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

$$R_2 \leftarrow R_2 - R_1$$

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

swap R_2 with R_2

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

next entry
to zero

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

Elimination Stage (Example)

next entry
to zero

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 3 & -6 & 6 & 4 & -5 \end{bmatrix}$$

$$R_3 \leftarrow R_3 - \frac{3R_2}{2}$$

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Elimination Stage (Example)

leftmost
nonzero
entry

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

swap R_3 with R_3

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Elimination Stage (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

done with elimination stage
going to back substitution stage

Back Substitution Stage

Back Substitution Stage (High Level)

Back Substitution Stage (High Level)

Input: matrix A of size $m \times n$ in echelon form

Output: reduced echelon form of A

Back Substitution Stage (High Level)

Input: matrix A of size $m \times n$ in echelon form

Output: reduced echelon form of A

Basic idea: scale pivot positions and eliminate the variable for that column from the other equations

Gaussian Elimination (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_1 \leftarrow R_1 / 3$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_2 \leftarrow R_2 / 2$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_1 \leftarrow R_1 + 3R_2$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_3 \leftarrow R_3 / 1$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_2 \leftarrow R_2 - R_3$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_1 \leftarrow R_1 - 5R_3$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 0 & -24 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 0 & -24 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

done with back substitution phase

Back Substitution (Psuedocode)

Back Substitution (Psuedocode)

```
FUNCTION back_sub(A):
```

Back Substitution (Psuedocode)

```
FUNCTION back_sub(A):
```

```
    FOR [i from 1 to m]: # for each row from top to bottom
```


Back Substitution (Psuedocode)

```
FUNCTION back_sub(A):
```

```
    FOR [i from 1 to m]: # for each row from top to bottom
```

```
        IF [row i has a leading entry]:
```

Back Substitution (Psuedocode)

FUNCTION back_sub(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [row i has a leading entry]:

$j \leftarrow$ index of leading entry of row i

Back Substitution (Psuedocode)

FUNCTION back_sub(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [row i has a leading entry]:

 j ← index of leading entry of row i

$R_i(A) \leftarrow R_i(A) / A[i, j]$ # divide by leading entry

Back Substitution (Psuedocode)

FUNCTION back_sub(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [row i has a leading entry]:

 j ← index of leading entry of row i

$R_i(A) \leftarrow R_i(A) / A[i, j]$ # divide by leading entry

FOR [k from 1 to i - 1]: # for the rows above the current one

Back Substitution (Psuedocode)

FUNCTION back_sub(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [row i has a leading entry]:

 j ← index of leading entry of row i

$R_i(A) \leftarrow R_i(A) / A[i, j]$ # divide by leading entry

FOR [k from 1 to i - 1]: # for the rows above the current one

$R_k(A) \leftarrow R_k(A) - R[k, j] * R_i(A)$

 # zero out R[k, j] above the leading entry

Back Substitution (Pseudocode)

FUNCTION back_sub(A):

FOR [i from 1 to m]: # for each row from top to bottom

IF [row i has a leading entry]:

$j \leftarrow$ index of leading entry of row i

$R_i(A) \leftarrow R_i(A) / A[i, j]$ # divide by leading entry

FOR [k from 1 to i - 1]: # for the rows above the current one

$R_k(A) \leftarrow R_k(A) - R[k, j] * R_i(A)$

 # zero out R[k, j] above the leading entry

RETURN A

Gaussian Elimination (Example)

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 3 & -9 & 12 & -9 & 6 & 15 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_1 \leftarrow R_1 / 3$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 2 & -4 & 4 & 2 & -6 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_2 \leftarrow R_2 / 2$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & -3 & 4 & -3 & 2 & 5 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_1 \leftarrow R_1 + 3R_2$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

pivot
position

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_3 \leftarrow R_3 / 1$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 1 & -3 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_2 \leftarrow R_2 - R_3$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

next entry
to zero

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 5 & -4 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

$$R_1 \leftarrow R_1 - 5R_3$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 0 & -24 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Gaussian Elimination (Example)

$$\begin{bmatrix} 1 & 0 & -2 & 3 & 0 & -24 \\ 0 & 1 & -2 & 2 & 0 & -7 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix}$$

done with back substitution phase

Gaussian Elimination (Example)

$$x_1 = (-24) + 2x_3 - 3x_4$$

$$x_2 = (-7) + 2x_3 - 2x_4$$

x_3 is free

x_4 is free

$$x_5 = 4$$

Recap: Solving a System of Linear Equations

Recap: Solving a System of Linear Equations

1. Write your system as an augmented matrix

Recap: Solving a System of Linear Equations

1. Write your system as an augmented matrix
2. Find the RREF of that matrix

Recap: Solving a System of Linear Equations

1. Write your system as an augmented matrix
2. Find the RREF of that matrix
3. Read off the solution from the RREF

Recap: Solving a System of Linear Equations

1. Write your system as an augmented matrix

2. Find the RREF of that matrix

Gaussian elimination

3. Read off the solution from the RREF

Numerics

demo

Significant Figures (Sig Figs)

Significant Figures (Sig Figs)

Do you remember sig figs from science class?

Significant Figures (Sig Figs)

Do you remember sig figs from science class?

Rulers only work up to $\pm 1\text{mm}$

Significant Figures (Sig Figs)

Do you remember sig figs from science class?

Rulers only work up to $\pm 1\text{mm}$

so we can't talk about nanometer differences

Significant Figures (Sig Figs)

Do you remember sig figs from science class?

Rulers only work up to $\pm 1\text{mm}$

so we can't talk about nanometer differences

We run into a similar problem with decimal numbers and computers

Number Representations

Number Representations

Your computer is a collection of fixed size registers

Number Representations

Your computer is a collection of fixed size registers

Each register holds a sequence of bits

Number Representations

Your computer is a collection of fixed size registers

Each register holds a sequence of bits

The Goal: represent numbers so they fit in those registers

Number Representations

Your computer is a collection of fixed size registers

Each register holds a sequence of bits

The Goal: represent numbers so they fit in those registers

this is, of course, ~~a lie~~ an abstraction

Number Representations

[illegible]

Number Representations

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Question. Which bits represent what?

Number Representations

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Question. Which bits represent what?

things to consider:

- » simple idea (easy to understand)
- » maximize coverage (not too redundant)
- » simple numeric operations (easy to use)

Unsigned Integers

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

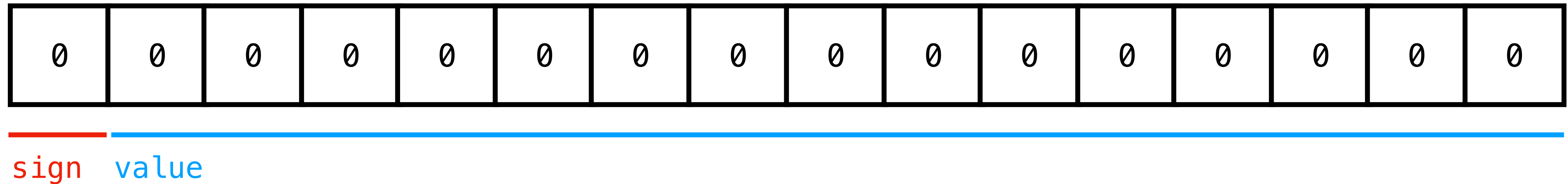
value

binary value (we should know this by now)

e.g. **1**000**1**0**1**0 represents

$$1(2^7) + 0(2^6) + 0(2^5) + 0(2^4) + 0(2^3) + 1(2^2) + 0(2^1) + 1(2^0)$$

Signed Integers



sign bit + binary value

e.g. **1**000**1010** represents

$$\text{−1} \times (0(2^6) + 0(2^5) + 0(2^4) + 0(2^3) + 1(2^2) + 0(2^1) + 1(2^0))$$

Floating-Point Numbers (Figures)

Floating-Point Numbers (Figures)

floats in python use *64 bits*

Floating-Point Numbers (Figures)

floats in python use *64 bits*

That's 1.8×10^{19} possible values

Floating-Point Numbers (Figures)

floats in python use *64 bits*

That's 1.8×10^{19} possible values

We can't represent everything. We'll have to choose and then round

Floating-Point Numbers (Figures)

floats in python use *64 bits*

That's 1.8×10^{19} possible values

We can't represent everything. We'll have to choose and then round

Question. Which ones should we represent?

Floating-Point Numbers (Idea)

Floating-Point Numbers (Idea)

Integers work because they are **discrete and evenly spaced**

Floating-Point Numbers (Idea)

Integers work because they are **discrete and evenly spaced**

What if we evenly discretize a range of values?

Floating-Point Numbers (Idea)

Integers work because they are **discrete and evenly spaced**

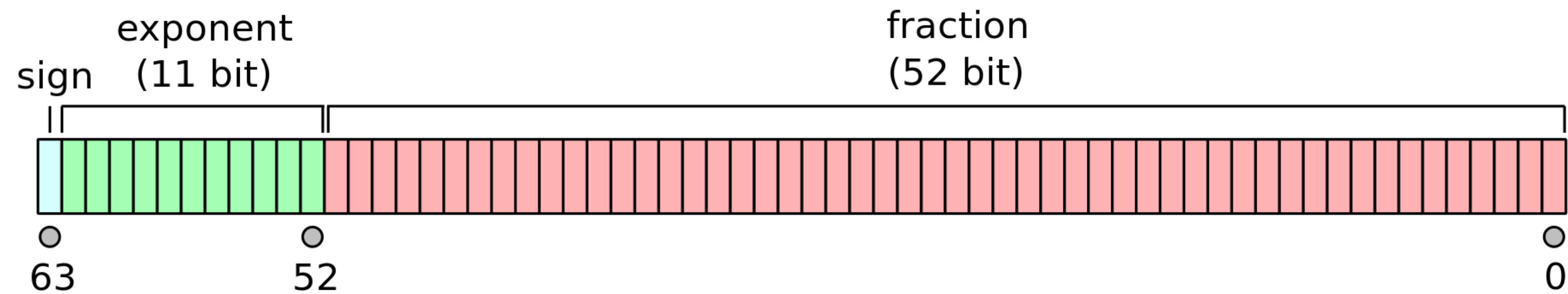
What if we evenly discretize a range of values?

i.e., represent

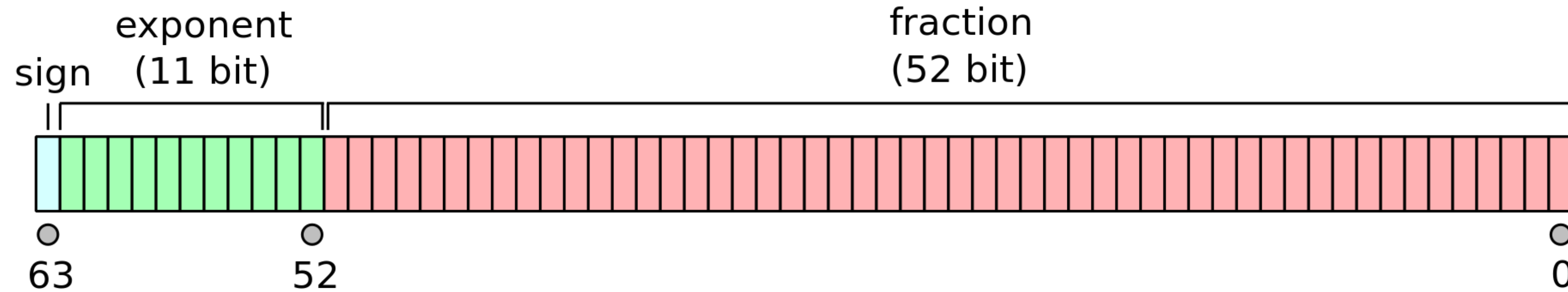
$\dots, -0.001, 0, 0.0001, 0.002, 0.003, 0.004, \dots$

Discuss the advantages and disadvantages of this approach

Floating-Point Numbers (IEEE-754)

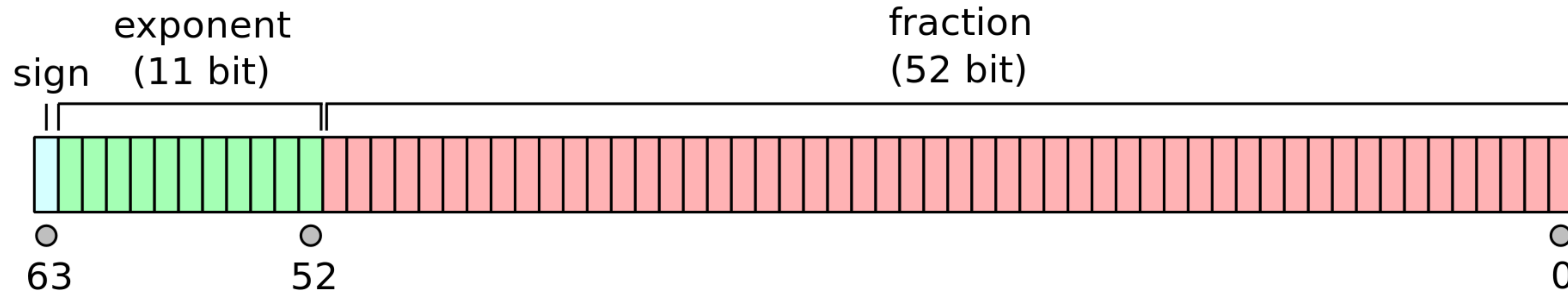


Floating-Point Numbers (IEEE-754)



This is like scientific notation, but binary:

Floating-Point Numbers (IEEE-754)



This is like scientific notation, but binary:

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}} \right) \times 2^{\text{exponent} - (2^{10} - 1)}$$

It's an accepted standard, not perfect, but it works well

Question

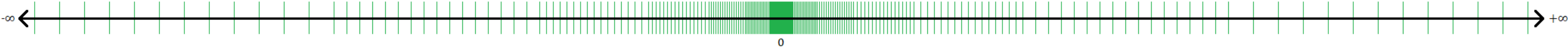
$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}}\right) \times 2^{\text{exponent} - (2^{10} - 1)}$$

Any ideas why this is better/worse?

And why not have a sign bit for the exponent?

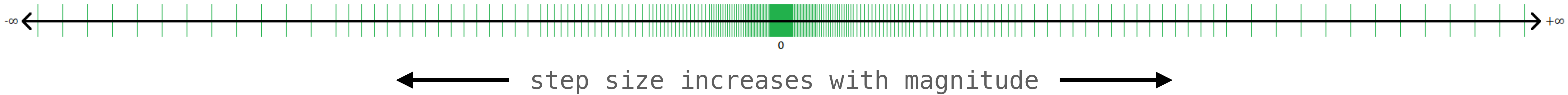
Step Size

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}} \right) \times 2^{\text{exponent} - (2^{10} - 1)}$$



Step Size

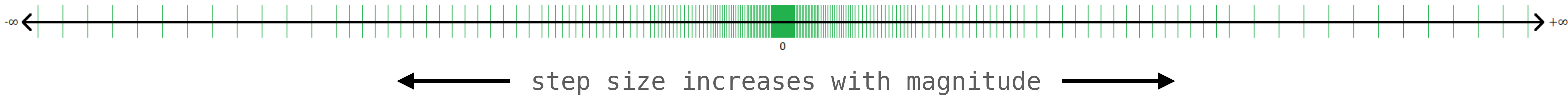
$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}} \right) \times 2^{\text{exponent} - (2^{10} - 1)}$$



Df. *step size* is the space between two floating-point representations

Step Size

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}} \right) \times 2^{\text{exponent} - (2^{10} - 1)}$$



Df. *step size* is the space between two floating-point representations

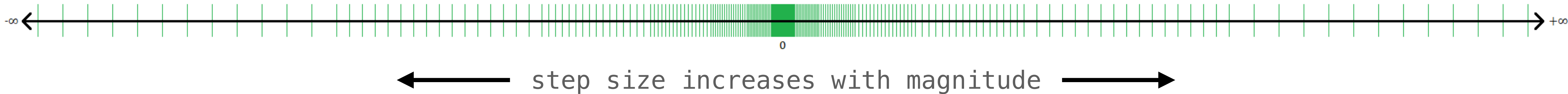
for fixed exponent n two numbers are at least

$$0.00\dots001 \times 2^n = 2^{-52} \times 2^n$$

away (why?)

Step Size

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}} \right) \times 2^{\text{exponent} - (2^{10} - 1)}$$



Df. *step size* is the space between two floating-point representations

for fixed exponent n two numbers are at least

$$0.00\dots001 \times 2^n = 2^{-52} \times 2^n$$

away (why?)

Step size doubles for each exponent

Things to Keep in Mind

Things to Keep in Mind

IEEE-754 defines a subset of decimal numbers

Things to Keep in Mind

IEEE-754 defines a subset of decimal numbers

operations on floating point numbers attempt to give you the closest to the actual value, though there will be errors

Things to Keep in Mind

IEEE-754 defines a subset of decimal numbers

operations on floating point numbers attempt to give you the closest to the actual value, though there will be errors

we can assume when we write down a number like '0.3' we get the closest IEEE-754 value

Relative Error

Observation. ± 0.001 is *tiny* error for 10^{20} but *massive* for 10^{-20}

Relative Error

Observation. ± 0.001 is *tiny* error for 10^{20} but *massive* for 10^{-20}

Relative Error.

$$\text{err}_{\text{rel}} = \frac{\text{err}}{\text{val}}$$

Relative Error

Observation. ± 0.001 is *tiny* error for 10^{20} but *massive* for 10^{-20}

Relative Error.

$$\text{err}_{\text{rel}} = \frac{\text{err}}{\text{val}}$$

IEEE-754 keeps relative error small

Relative Error (Calculation)

(fix an exponent n)

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}} \right) \times 2^{\text{exponent} - (2^{10} - 1)}$$

Relative Error (Calculation)

(fix an exponent n)

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}}\right) \times 2^{\text{exponent} - (2^{10} - 1)}$$

error is determined by step-size

$$\text{err} \leq 2^{-52} \times 2^n$$

Relative Error (Calculation)

(fix an exponent n)

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}}\right) \times 2^{\text{exponent} - (2^{10} - 1)}$$

the smallest number we can represent is 1.0×2^n

$$\text{val} \geq 1.0 \times 2^n$$

(why do we care about a lower bound on val?)

Relative Error (Calculation)

(fix an exponent n)

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}} \right) \times 2^{\text{exponent} - (2^{10} - 1)}$$

Relative Error (Calculation)

(fix an exponent n)

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}}\right) \times 2^{\text{exponent} - (2^{10} - 1)}$$

the relative error is *small*

$$\text{val} \geq 1.0 \times 2^n$$

$$\text{err} \leq 2^{-52} \times 2^n$$

Relative Error (Calculation)

(fix an exponent n)

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}}\right) \times 2^{\text{exponent} - (2^{10} - 1)}$$

the relative error is *small*

$$\text{val} \geq 1.0 \times 2^n$$

$$\text{err} \leq 2^{-52} \times 2^n$$

$$\text{err}_{\text{rel}} = \frac{\text{err}}{\text{val}} \leq \frac{2^{-52} \times 2^n}{1.0 \times 2^n} = 2^{-52} \approx 10^{-16}$$

Relative Error (Calculation)

(fix an exponent n)

$$(-1)^{\text{sign}} \times \left(1 + \frac{\text{fraction}}{2^{52}}\right) \times 2^{\text{exponent} - (2^{10} - 1)}$$

the relative error is *small*

$$\text{val} \geq 1.0 \times 2^n$$

$$\text{err} \leq 2^{-52} \times 2^n$$

$$\text{err}_{\text{rel}} = \frac{\text{err}}{\text{val}} \leq \frac{2^{-52} \times 2^n}{1.0 \times 2^n} = 2^{-52} \approx 10^{-16}$$

≈ 16 digits of accuracy

Not bad, but also not great

demo

(example from the notes)

The Takeaways

The Takeaways

operations on floating-points are not exact

The Takeaways

operations on floating-points are not exact

properties like $(ab)c = a(bc)$ may not hold

The Takeaways

operations on floating-points are not exact

properties like $(ab)c = a(bc)$ may not hold

it's a trade-off for large range and low
relative error

The Takeaways

operations on floating-points are not exact

properties like $(ab)c = a(bc)$ may not hold

it's a trade-off for large range and low relative error

What can we do about it?

Principle 1: Closeness

Principle 1: Closeness

When doing floating-point calculations in a program, define an error margin and use that for equality checking

Principle 1: Closeness

When doing floating-point calculations in a program, define an error margin and use that for equality checking

In Practice.

Replace
with

```
x == y  
numpy.isclose(x, y)
```


demo

Principle 2: Small Differences

Principle 2: Small Differences

*Make sure you understand your error tolerance
when looking at the small differences of
large numbers*

Principle 2: Small Differences

Make sure you understand your error tolerance when looking that the small differences of large numbers

In Practice. Don't expect $a - b$ to be small when a and b are "close" but very large.

demo

Principle 3: Ill-Conditioned Problems

Principle 3: Ill-Conditioned Problems

Make sure your problem is not sensitive to small errors

Principle 3: Ill-Conditioned Problems

Make sure your problem is not sensitive to small errors

In Practice. for example, don't divide by numbers much smaller than your error tolerance

demo

One Last Note: Special Numbers

`0` (we can't already represent 0?)

`nan` stands for not a number, .e.g, `sqrt(-2)`

`inf` symbolic infinity, behaves as expected

Extra Topic: Analyzing the Algorithm

Analyzing the Algorithm

Analyzing the Algorithm

We will not use $O(\cdot)$ notation!

Analyzing the Algorithm

We will not use $O(\cdot)$ notation!

For numerics, we care about number of **FLoating-point
OPerations** (FLOPs):

- >> addition
- >> subtraction
- >> multiplication
- >> division
- >> square root

Analyzing the Algorithm

We will not use $O(\cdot)$ notation!

For numerics, we care about number of **F**loating-oint
Operations (FLOPs):

- >> addition
- >> subtraction
- >> multiplication
- >> division
- >> square root

$2n$ vs. n is very different
when $n \sim 10^{20}$

Dominant Terms

Dominant Terms

that said, we don't care about *exact* bounds

Dominant Terms

that said, we don't care about *exact* bounds

A function $f(n)$ is ***asymptotically equivalent*** to $g(n)$ if

$$\lim_{i \rightarrow \infty} \frac{f(i)}{g(i)} = 1$$

Dominant Terms

that said, we don't care about *exact* bounds

A function $f(n)$ is ***asymptotically equivalent*** to $g(n)$ if

$$\lim_{i \rightarrow \infty} \frac{f(i)}{g(i)} = 1$$

for polynomials, they are equivalent to their dominant term

Dominant Terms

The **dominant term** of a polynomial is the monomial with the highest degree

$$\lim_{i \rightarrow \infty} \frac{3x^3 + 100000x^2}{3x^3} = 1$$

$3x^3$ dominates the function even though the coefficient for x^2 is so large

Parameters

n : number of variables

m : number of equations (we will assume $m = n$)

$n + 1$: number of rows in the augmented matrix

The Cost of a Row Operation

$$R_i \leftarrow R_i + aR_j$$

$n + 1$ multiplications for the scaling

$n + 1$ additions for the row additions

Tally: $2(n + 1)$ FLOPS

Cost of First Iteration of Elimination

$$R_2 \leftarrow R_2 + a_2 R_1$$

$$R_3 \leftarrow R_3 + a_3 R_1$$

$$\vdots$$

$$R_n \leftarrow R_n + a_n R_1$$

Repeated row operations for each row except the first

Tally: $\approx 2n(n+1)$ FLOPS

Rough Cost of Elimination

repeating this last process at most n times
gives us a dominant term $2n^3$

we can give a better estimation...

Tally: $\approx 2n^2(n + 1)$ FLOPS

Cost of Elimination

0	■	*	*	*	*	*	*	*	*
0	0	0	■	*	*	*	*	*	*
0	0	0	0	*	*	*	*	*	*
0	0	0	0	*	*	*	*	*	*
0	0	0	0	*	*	*	*	*	*
0	0	0	0	0	0	0	0	0	0

At iteration i , we're only interested in rows after i

And to the right of column i

Cost of Elimination

Iteration 1: $2n(n+1)$

Iteration 2: $2(n-1)n$

Iteration 3: $2(n-2)(n-1)$ +
⋮

$$\sum_{k=1}^n 2k(k+1) \approx \frac{2n(n+1)(2n+1)}{6} \sim (2/3)n^3$$

Tally: $\sim (2/3)n^3$ FLOPS

Cost of Back Substitution

Assume no free variables, for each pivot, we only need to:

- » zero out a position in 1 row (0 FLOPS)

- » add a value to the last row (1 FLOP)

at most 1 FLOP per row per pivot $\sim n^2$

Tally: $\sim (2/3)n^3$ FLOPS

Cost of Gaussian Elimination

Tally: $\sim (2/3)n^3$ FLOPS

(dominated by elimination)

Summary

Gaussian elimination is a codification of forward elimination and back-substitution

Decimal numbers are represented in your computer as **floating points**, which can result in error when doing computations

The running time of Gaussian elimination is
 $\sim (2/3)n^3$ FLOPS